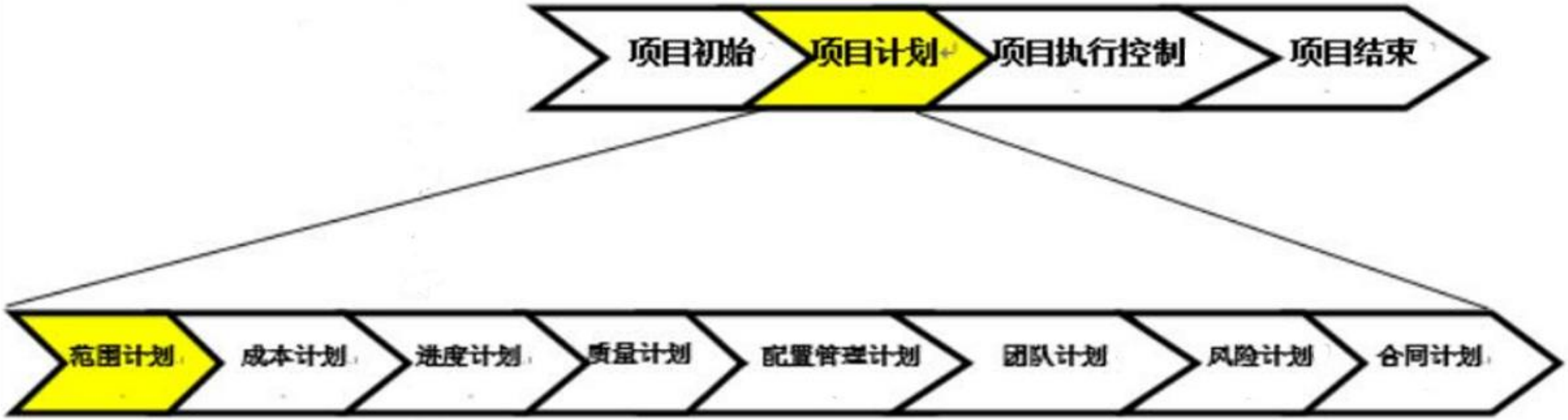
IT项目管理

史建锋

2025年11月17日


项目初始
项目计划
项目执行控制
项目结束
范围计划
成本计划
进度计划
质量计划
配置管理计划
团队计划
风险计划
合同计划

- **项目范围管理**是对项目包括什么与不包括什么的定义与控制过程，确定项目范围是制定IT项目计划的根据，包括对功能、性能、接口和可靠性的确定。通过项目范围管理，可以明确项目管理的目标与边界。
- 需求是IT项目建设的基石。资料表明，IT项目中40-60%的问题都是在需求分析阶段埋下的隐患。软件项目中40%的开销用于返工，而其中的70-80%是需求错误所致。

第 4 章

IT项目需求管理

本 章 要 点

一 IT项目需求管理过程

二 传统需求建模方法

三 敏捷需求建模方法

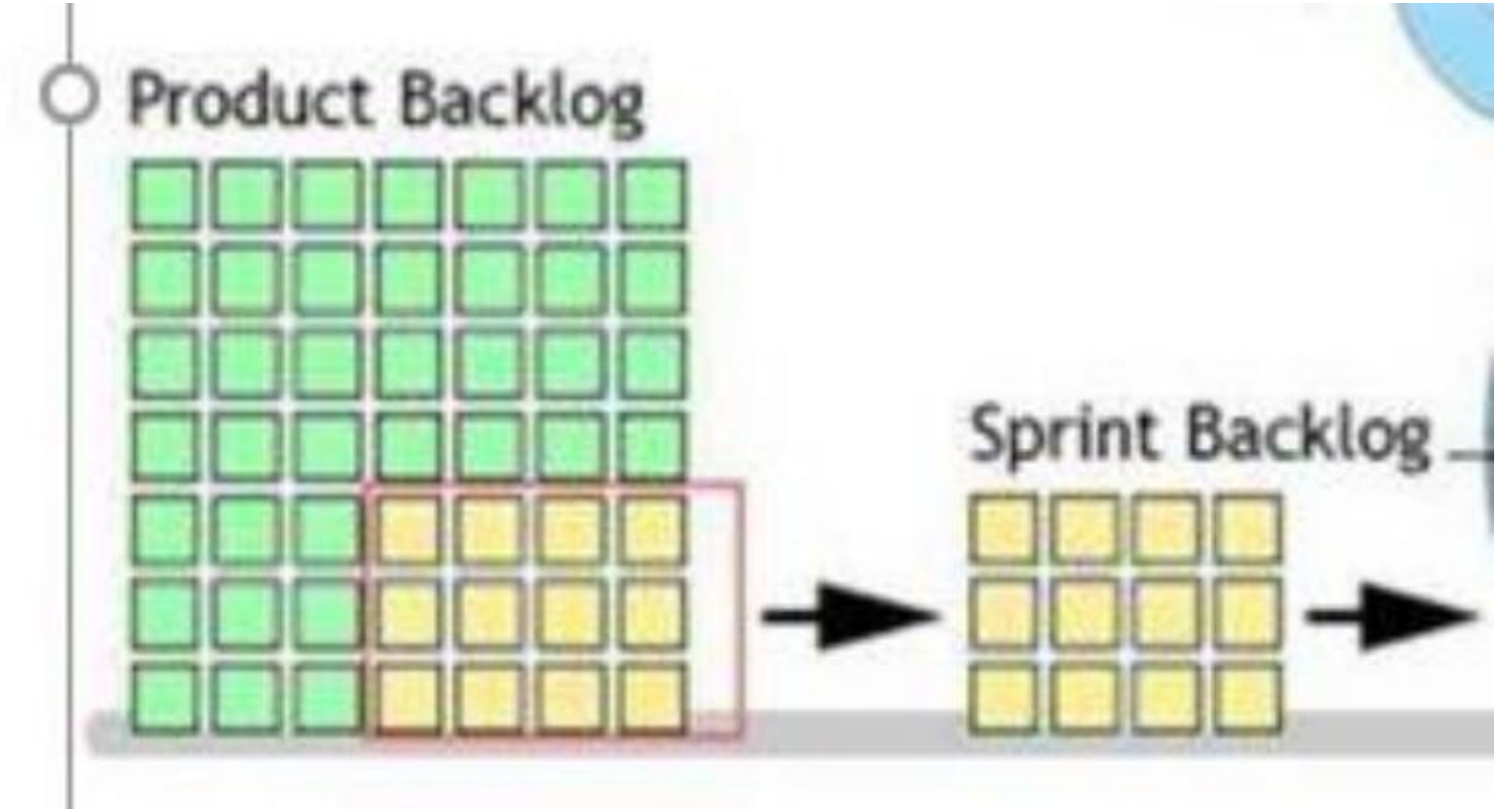
四 AI驱动的需求建模

五 案例分析

敏捷项目需求建模方法（工具）

- 需求池（Backlog）
- 用户故事（User story）
  - 用户故事的优先级排序MoSCoW
  - 何谓好的用户故事INVEST
- 行为驱动开发(BDD)

需求池Backlog



Product Backlog：产品待办事项列表

- 需求建模的来源
- 包含产品想法的一个有序列表
- 一个长短不定列表
- 可以是模糊的或是不具体的
- 逐渐完善，越来越明确

用 户 故 事 描 述

- 敏捷开发过程中，用户需求经常被描述为用户故事（User Story）。
- 用户故事，是从用户的角度来描述用户渴望得到的功能，一个好的用户故事包括三个要素：

1. 角色：谁要使用这个功能；
2. 活动：需要完成什么样的功能；
3. 商业价值：为什么需要这个功能，这个功能带来什么样的价值。

用 户 故 事 描 述

注：尽管和用例功能类似。但用户故事并不等同于传统建模方法中的用例，用例是一种详细描述系统如何响应外部交互以实现特定目标的方法。它通常包括参与者、事件流、前置条件和后置条件等元素，详细说明了用户与系统交互的每个步骤。用例在传统的瀑布模型开发过程中使用较多，确保了需求的完整性和系统的复杂交互。

用户故事是敏捷开发中常用的一种需求描述方法，强调简洁性和从用户角度出发。用户故事通常以自然语言编写，格式为“作为一个[角色]，我想[目标]，以便[原因]”，强调用户需求背后的目的和价值，而不是具体的实现细节。

用户故事编写

User stories are short, simple description of a feature.They typically follow a simple template:

- As a <*type of user*>,
- I want <*some goal*>
- so that <*some reason*>.

半结构化

用户故事编写

- User stories examples:

> As a user,
> I want to backup my entire hard drive
> so that I won't lose any work.

用户故事编写

Story card template (front)

Story name	Story ID
As a I want So that	
Priority high 1 2 3 4 5 low Date Started	Iteration number Date Finished

用户故事编写

Story card template (back)

Acceptance Criteria
Notes

用户故事编写，验收标准（AC）

Story name	**create account** **Story ID E01c**
As a	**Bank Employee**
I want to create an account of the type chosen for an existing customer, generate a unique account number and PIN.	
我想为现有客户创建一个所选类型的账户，生成一个唯一的账户号码和PIN码。	
So that I can open the account and notify customer of account number and PIN.	
以便我可以开通账户，并通知客户账户号码和PIN码	
Priority high 1 2 3 4 5 low	**Iteration number**
Date Started	**Date Finished**

用户故事编写，验收标准（AC）

Acceptance Criteria

验证系统中是否存在匹配给定详情的客户。
确保已将客户的信用状况记录为满意。

- Verify that customer exists in system matching details given
- Make sure that customer's credit status has been recorded as satisfactory
- Verify that account of required type is created
- Verify that account number is unique
- Verify that PIN is 6 digits

Notes

用户故事中的非功能性需求

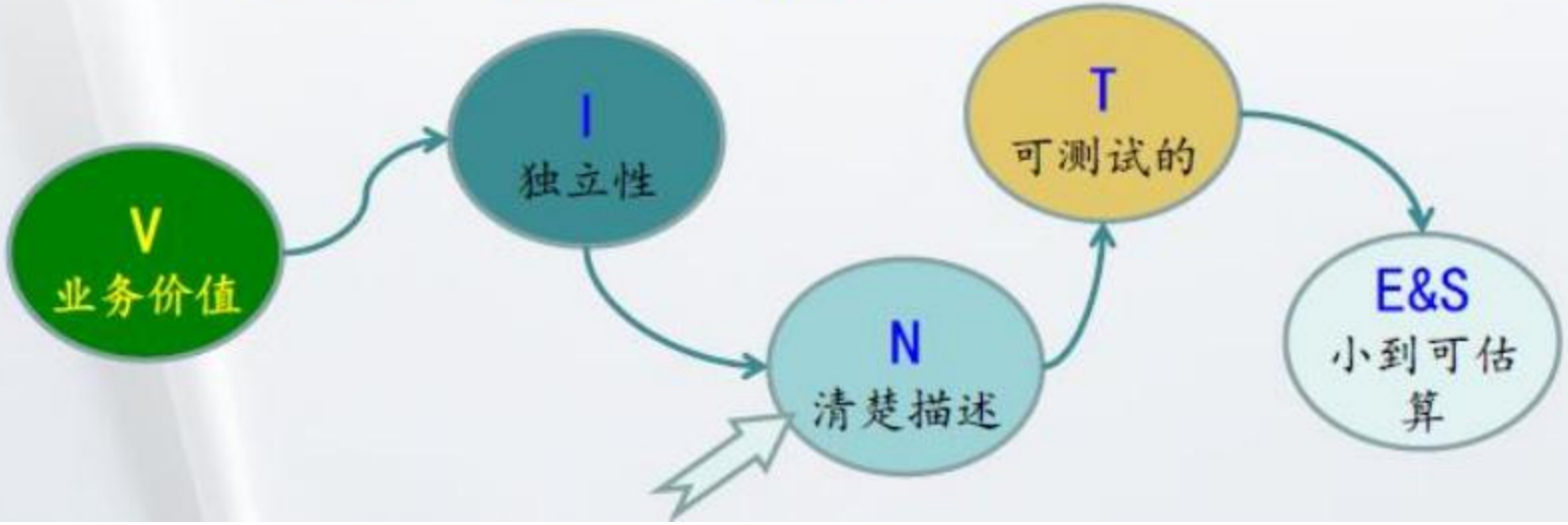
Non-functional Requirements as User Stories

- Non-functional requirements (constraints) can also be handled as user stories. Examples:
  - As a customer, I want to be able to run your product on all versions of Windows from Windows XP on, so that we can make use of older PCs. 兼容性---操作系统
  - As the CTO, I want the system to use our existing orders database rather than create a new one, so that we don't have one more database to maintain. 兼容性---运行环境
  - As a user, I want the site to be available 99% of the time I try to access it, so that I don't get frustrated and find another site to use. 鲁棒性-健壮性
  - As someone who speaks Chinese, I want to have a Chinese version of the software.兼容性—语言
- If you can't find a way to word the constraint, just write the constraint in whatever way feels natural.

好的用户故事

Given <condition>, When <Time/Event>, Then <Results>

- 六个特性：INVEST - Independent, Negotiable, Valuable, Estimable, Small, Testable

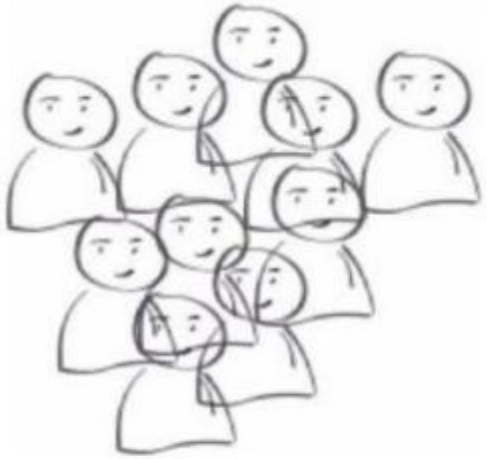


用户故事必须进行优先级排序。

- 基于业务价值
- 价值必须得到投资回报率（ROI）的支持
- 风险评估

客户根据他们的优先级和进度估计来选择包含在下一次发布中的故事。

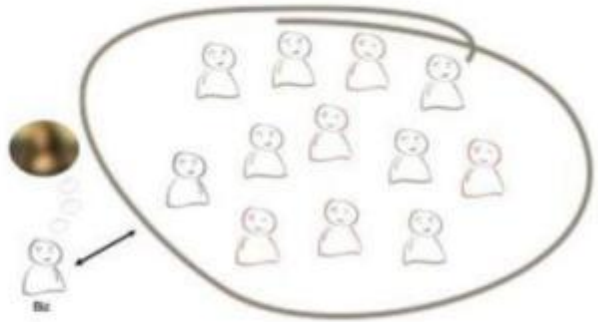
价值优先



增加用户数量

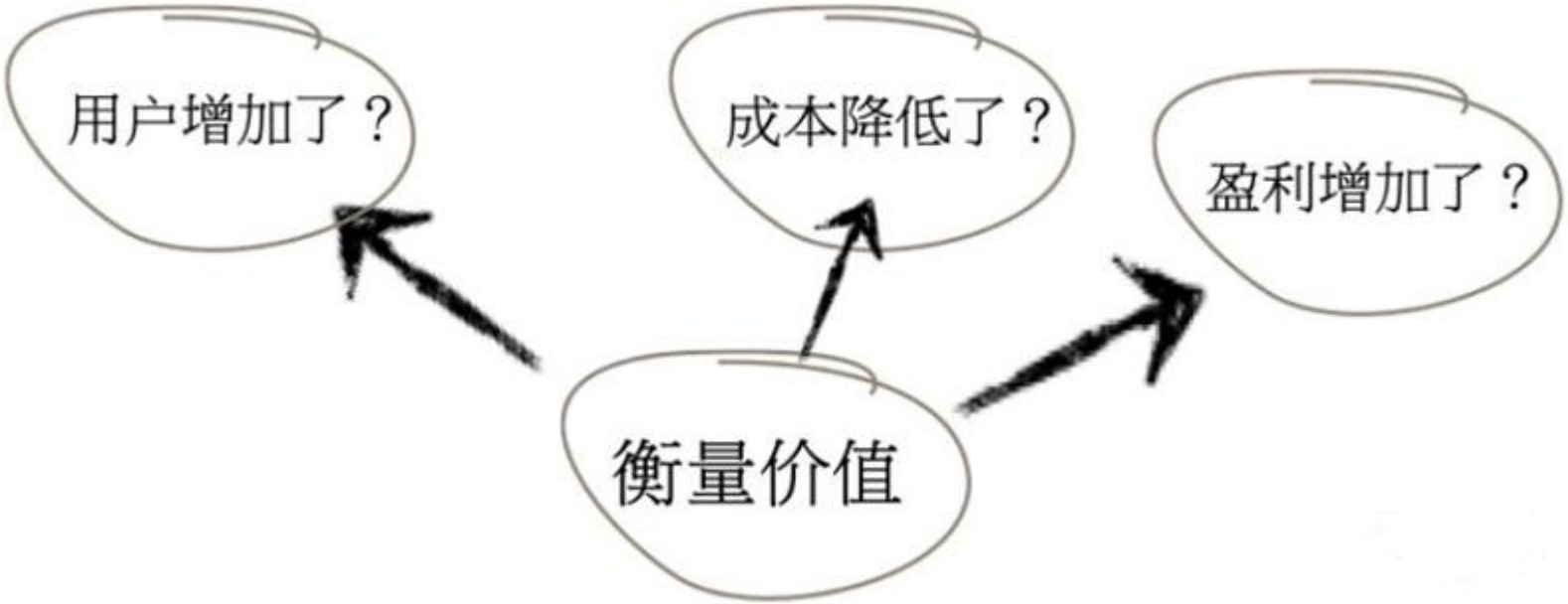


增加营收 & 减少成本



更好地服务于更多人

衡量价值的维度



MoSCoW

- A method of prioritisation favoured by the DSDM (dynamic systems development method). Elements are:

必须有 – Must have: features must be implemented, and if not, the system would not work. 这些功能必须实现，否则系统将无法运行。

应该有 – Should have: features are important but can be omitted if time or resources constraints appear. 这些功能很重要，但若出现时间或资源限制，可以省略。

可以有 – Could have: features enhance the system with greater functionality, but the timelines of their delivery is not critical. 这些功能能提升系统性能，但其交付时间并不关键。

能有更好 – Want to have: features serve only a limited group of users and do not drive the same amount of business value as the preceding items. 这些功能仅服务于有限用户群体，且带来的业务价值不如前几类。

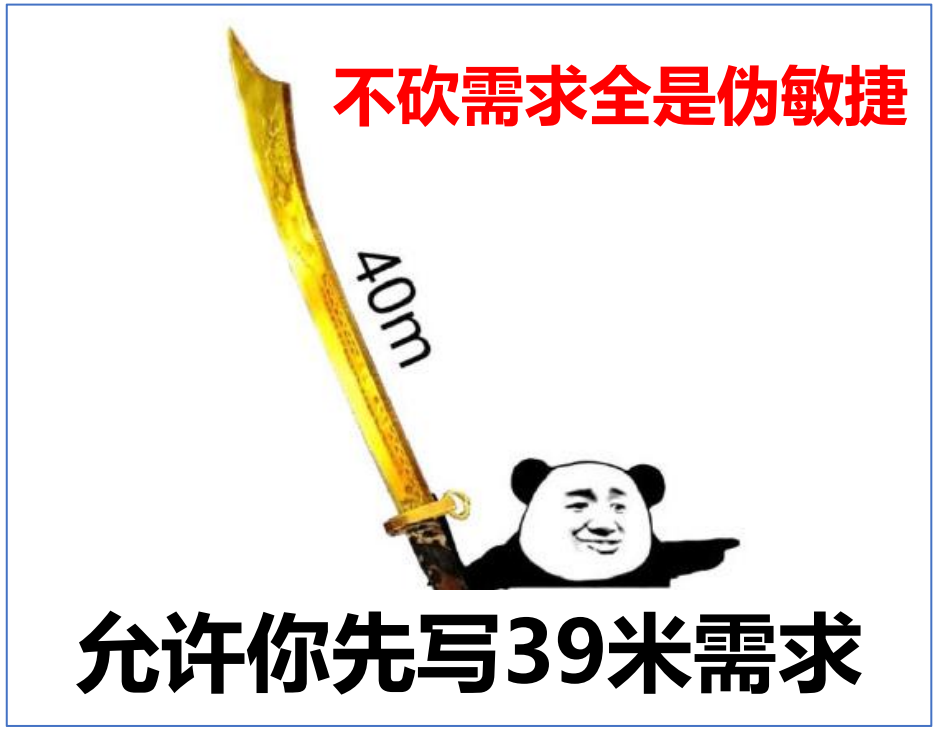
用 户 故 事 描 述

MoSCoW优先级排序法	
Must	完成业务需要/价值所必须的功能、合规、安全等项
Should	不涉及核心功能，但是严重影响用户体验
Could	影响不大，忽略不做也没问题
Won’t	团队决定本次迭代不予完成



例：Payment processing story:

- –Must have: ability to accept Visa and MasterCard.
- –Should have: add American Express.
- –Could have: add PayPal.
- –Want to have: add gift cards.



真正使用的需求很少

根据美国著名产品分析平台Pendo在2019年对600多个产品的使用监测调研，80%的功能很少或者从未被使用过。由此造成的浪费接近300亿美金



— Pendo.io 《2019 Feature Adoption Report》

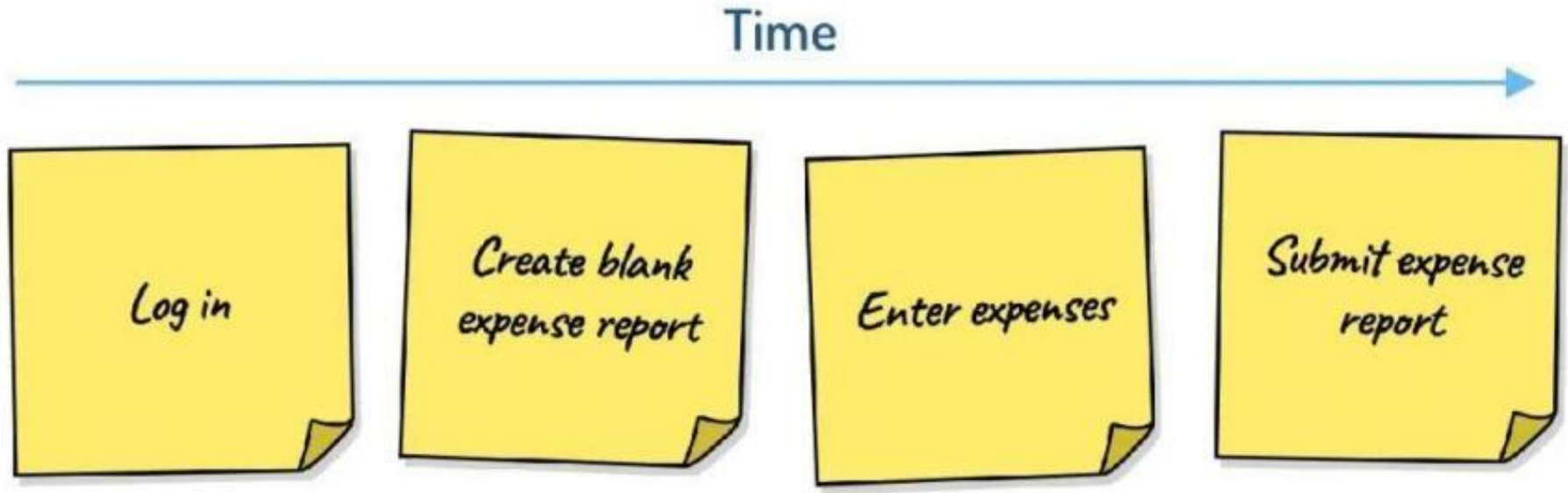
用 户 故 事 地 图



故事地图可以帮助团队发现他们可能忽略的用户活动或功能。

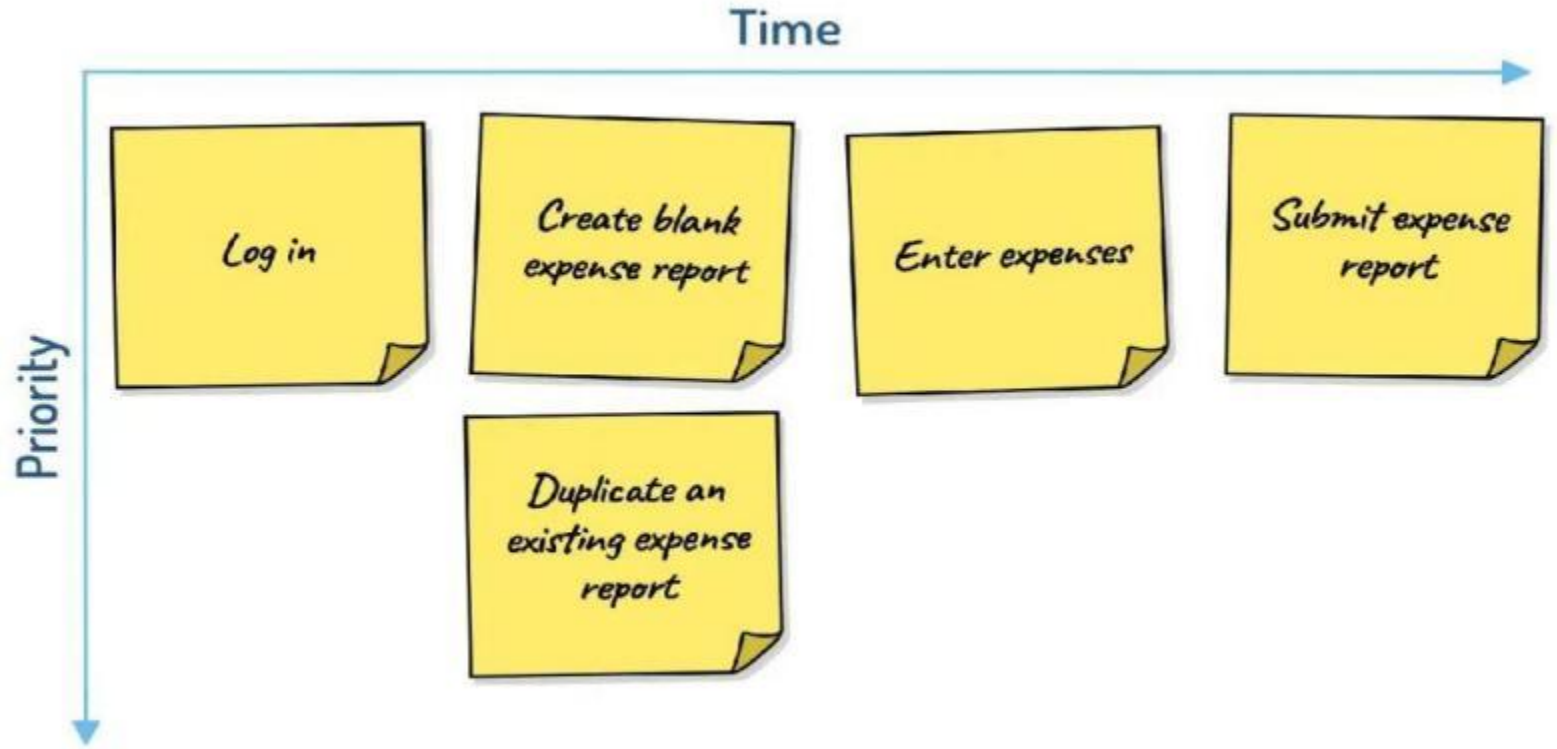
第一维度：横向

故事地图的水平维度，是在地图上水平展开的一系列用户任务（功能），代表了一个时间顺序。



第二维度：纵向

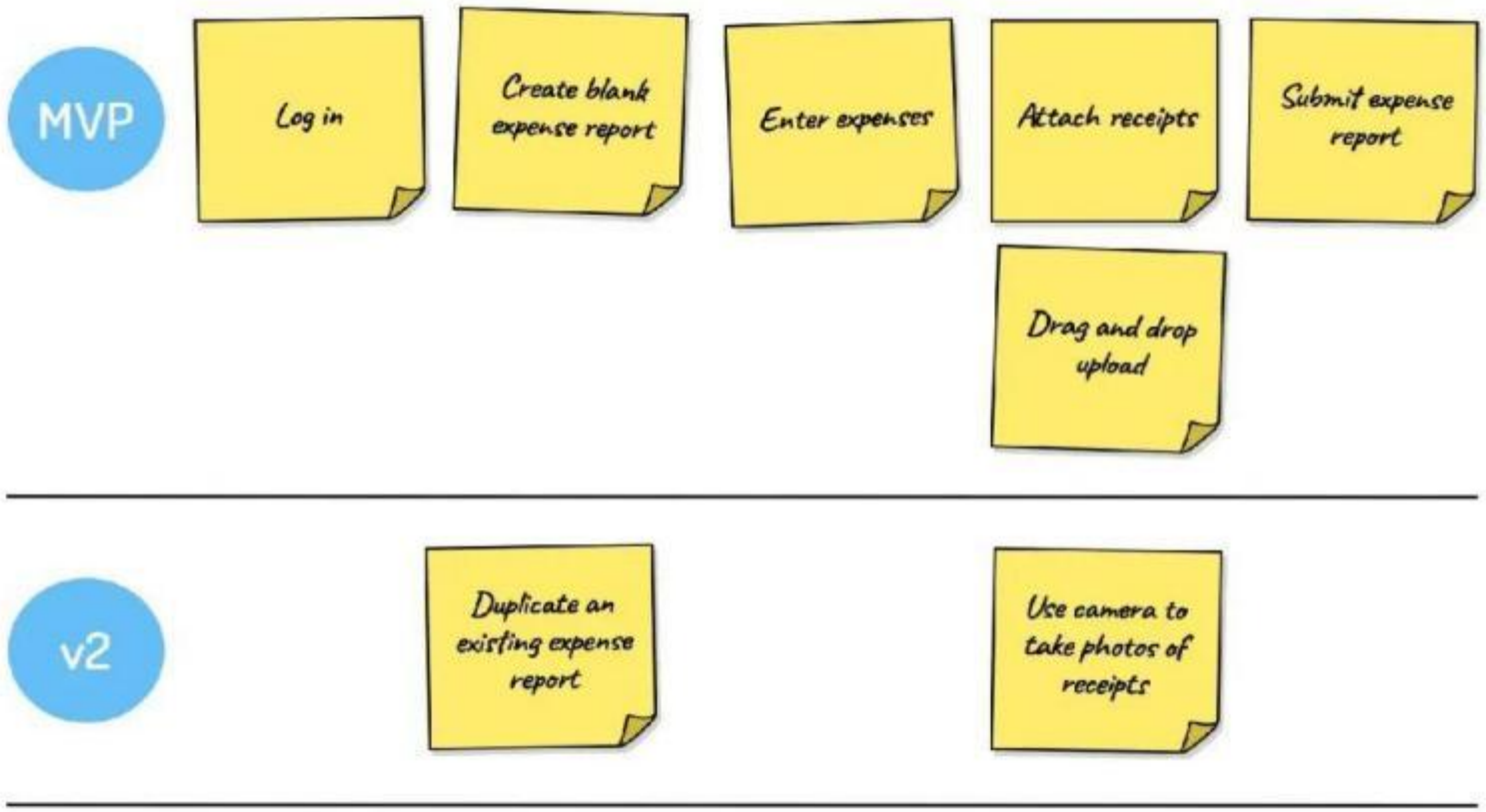
故事地图第二维可以理解为本功能（任务）的其他方案。



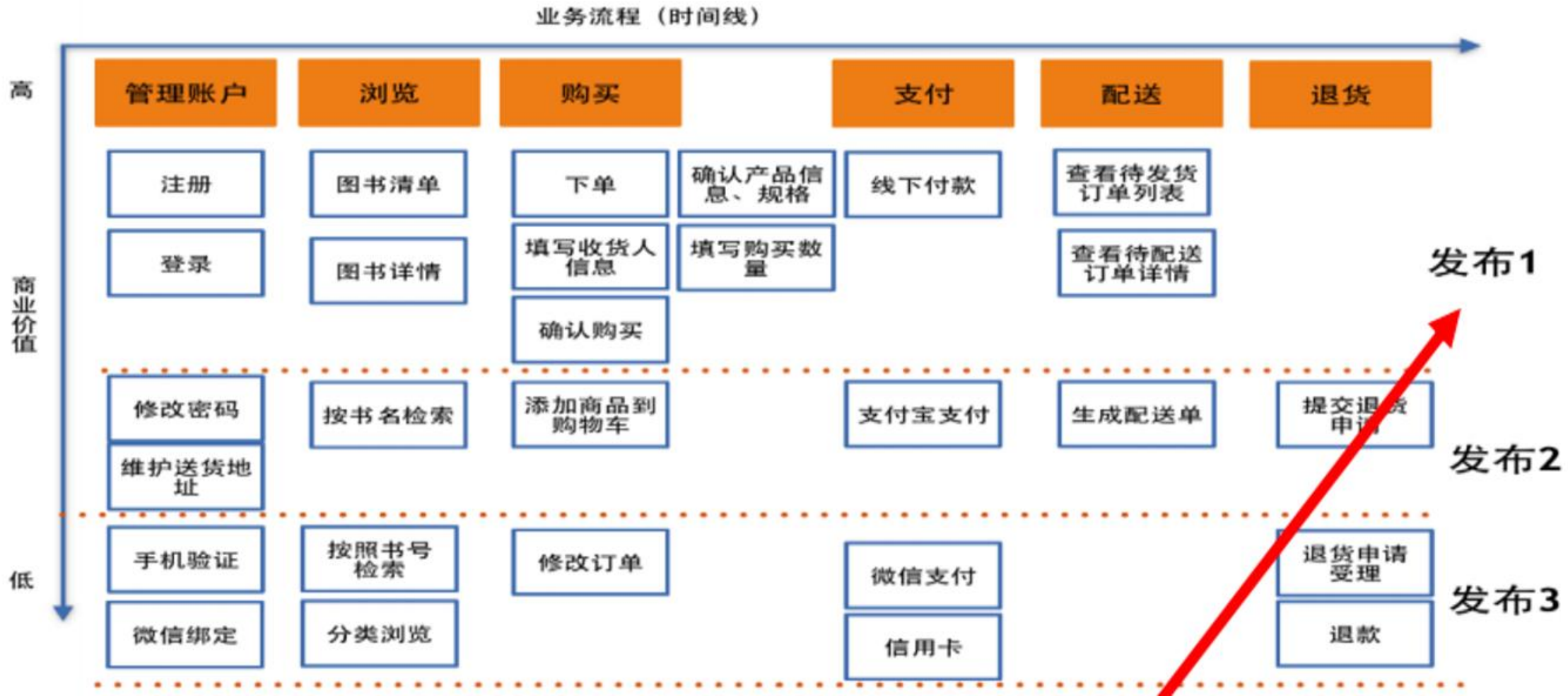
故事地图有助于发现缺失的功能。



使用故事地图可视化发布



示例：在线购书网站



MVP（Minimum Viable Product）：最小化可行产品

故事卡片与故事墙

- User stories are often written on index cards or sticky notes, stored in a shoe box, and arranged on walls or tables to facilitate planning and discussion.

用户故事通常写在索引卡或便签上，存放在盒子里，并摆放在墙上或桌子上，以方便规划和讨论。

- As such, they strongly shift the focus from writing about features to discussing them. In fact, these discussions are more important than whatever text is written.

因此，它们将重点从撰写功能描述转变为讨论功能。实际上，这些讨论比写下的任何文本都要重要。



Story Wall



- 行为驱动开发（Behavior-Driven Development，简称BDD）是一种敏捷软件开发的技术，它鼓励软件项目中的开发者、业务分析师和测试人员之间的协作。BDD的核心思想是通过与利益相关者的协作，以行为为导向来定义、设计和开发软件。
- BDD是用户故事的“扩展与落地”，通过具体场景和验收标准确保需求可执行、可测试。两者结合能有效弥合业务语言与技术实现之间的鸿沟，提升需求分析的准确性和可交付性。

行为驱动开发的主要特点和步骤：

1.沟通和协作：BDD强调团队成员之间的沟通，确保每个人对软件应该展现的行为有共同的理解。
2.定义行为：使用一种特定的、易于理解的语言来描述软件的行为。这些描述通常采用“给定（Given）- 当（When）- 那么（Then）”的格式，这种格式也被称为Gherkin语言。
3.实例化需求：将需求转化为具体的例子，这些例子可以作为验收测试的依据。
4.自动化测试：基于定义的行为实例，编写自动化测试脚本。这些测试脚本不仅用于验证代码的正确性，还可以作为文档来描述系统的行为。
5.开发：在编写代码时，开发者会参考这些行为描述和测试案例，确保他们的实现满足预定的行为。
6.反馈循环：BDD是一个迭代过程，其中包括持续的反馈和改进。团队成员会根据测试结果和用户反馈来调整和优化软件行为。

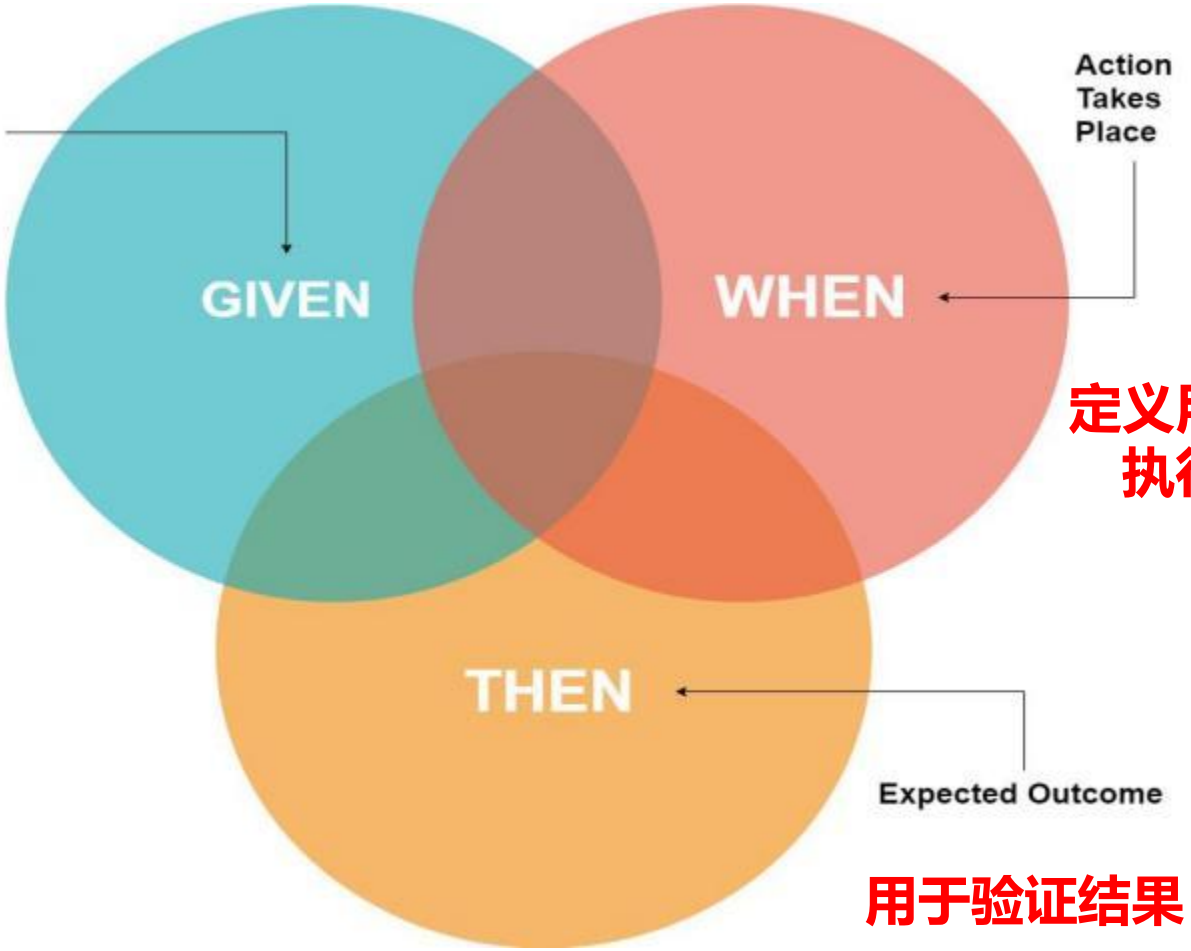
Given-When-Then

BDD使用一种简单的语法格式称为Gherkin

GIVEN <a situation>
WHEN <an event>
THEN <expected result>

BDD（Behavior Driven Development行为驱动开发）

实例化需求的工具



设定测试的预置条件

定义用户或系统执行的动作

用于验证结果

场景实例：加法

```
Feature: Calculator
  As a user
  I want to use a calculator to add numbers
  So that I don't need to add myself

  Scenario: Add two numbers -2 & 3
    Given I have a calculator
    When I add -2 and 3
    Then the result should be 1

  Scenario: Add two numbers 10 & 15
    Given I have a calculator
    When I add 10 and 15
    Then the result should be 25
```

Scrum中文网

User story vs BDD

•焦点：用户故事关注用户的需求和功能，而BDD关注软件的行为和业务规则。

•详细程度：用户故事通常比较抽象，而BDD的场景更具体，更接近测试用例。

•用途：用户故事是需求的表达，而BDD是一种开发方法，它可以使用用户故事作为输入来编写具体的测试场景。

•格式：用户故事有特定的格式，而BDD的场景通常遵循Given-When-Then的格式。

用户故事是BDD过程中的一个组成部分，它们可以被视为BDD的“原材料”。BDD使用用户故事来定义和验证软件的行为，确保最终产品符合用户的实际需求和业务目标。

本 章 要 点

一 IT项目需求管理过程

二 传统需求建模方法

三 敏捷需求建模方法

四 AI驱动的需求建模

五 案例分析

借助AI技术，未来可以实现业务人员和AIGC的直接对话，就能生成业务所需的软件，**不是经过程序员这层“翻译”。**

借助大模型，可以帮助快速梳理思路。

- 梳理业务流程

3 做一个在线点餐APP，用户下单的全流程应该如何?

用户打开APP后，首先进入主页，选择需要订购的餐厅，并进入该店铺的主页。在该店铺主页中，用户可以选择菜品并加入购物车，以及设置数量、备注等信息。当用户已经添加完所有需要的菜品后，可选择结算并进入支付页面。在支付页面，用户需要输入配送地址、联系电话等信息，选择支付方式，并完成支付。系统接收到用户的订单后会发送推送消息给店家，提餐员会送餐到用户指定的地址。用户可以在APP中查看订单状态、历史订单等信息。

借助大模型，可以帮助快速梳理思路。

- 编写用户故事

3 你懂用户故事吗？他的格式是 作为一个xxx, 我想要xxx,以便于xxx, 你能把上面内容拆解成多个用户故事吗？

当然可以，以下是一些可能的用户故事：

1. 作为一个餐品爱好者，我想浏览店铺中的菜品，以便于发现我喜欢的新鲜美食。
2. 作为一个忙碌的上班族，我想在工作时间预订餐品，以便于在下班后节省时间去吃晚饭。
3. 作为一个对口味有特殊要求的食客，我想对菜品进行备注，以便于餐厅准确制作我所需要的菜品。
4. 作为一个经常下单的用户，我想能够查看历史订单，以便于方便地再次下单。
5. 作为一个节约成本的用户，我想查看菜品的价格等信息，以便于选择性价比最高的餐品。
6. 作为一个喜欢在线支付的用户，我想选择支付方式并完成支付，以便于订单能够快速处理。

借助大模型，可以帮助快速梳理思路。

- 设定用户故事角色

3 请把上面的用户故事，统一用一个用户角色来替代，譬如 吃货

好的，下面是以"吃货"为用户角色的用户故事：

1. 吃货想要在APP中浏览店铺的菜品，以便于发现新鲜美食。
2. 吃货想在APP中预订餐品，以便于在下班后节省时间去吃晚饭。
3. 吃货想在APP中添加菜品备注，以便于餐厅准确制作他所需要的菜品。
4. 吃货想查看自己在APP中的历史订单，方便再次下单。
5. 吃货想在APP上查看菜品的价格等信息，以便于选择性价比最高的餐品。
6. 吃货想在APP上选择支付方式并完成支付，以便于订单能够快速处理。

借助大模型，可以帮助快速梳理思路。

- 用户故事拆解

3 请把上面的每一个用户故事，分别进行拆解细化成多个细化的用户故事，并分别进行标记，区分出来层次感，譬如1.1，1.2，1.3 等编号，是对第一个用户故事的拆解，以此类推

好的，下面是对每个用户故事的细化拆解，每个拆解后的用户故事前面带有该故事对应的编号：

1. 吃货想要在APP中浏览店铺的菜品，以便于发现新鲜美食。 1.1 吃货希望能够按照菜品种类筛选菜品，以便于看到同一种类菜品的分类展示。 1.2 吃货希望能够按照店家推荐、新品等方式查看菜品，以便于快速发现店家推荐的餐品。 1.3 吃货希望在查看菜品时能够看到每个菜品的图片、介绍等详细信息，以便于了解菜品的制作方式和口感特点。
2. 吃货想在APP中预订餐品，以便于在下班后节省时间去吃晚饭。 2.1 吃货希望能够选定餐品的数量，以便于一次下单多份餐品。 2.2 吃货希望APP能够提供预估配送时间的功能，以便于可以计划好自己的时间。 2.3 吃货希望APP提供设置默认配送地址的功能，以便加快预订下单的速度。
3. 吃货想在APP中添加菜品备注，以便于餐厅准确制作他所需要的菜品。 3.1 吃货希望能够在APP上添加菜品的口味偏好、餐品设置等信息，以便于确保菜品符合他的特殊喜好。 3.2 吃货希望APP上能够提供添加备注、修改菜品信息或删除菜品等功能，以便于方便地进行菜品修改和调整。
4. 吃货想查看自己在APP中的历史订单，方便再次下单。 4.1 吃货希望在APP上浏览自己的历史订单，并能够按时间、金额等不同方式进行排序和筛选。 4.2 吃货希望在历史订单页面中能方便地再次进行下单、加入购物车等操作。
5. 吃货想在APP上查看菜品的价格等信息，以便于选择性价比最高的餐品。 5.1 吃货希望在APP中能够显示每个菜品的价格

本 章 要 点

一 IT项目需求管理过程

二 传统需求建模方法

三 敏捷需求建模方法

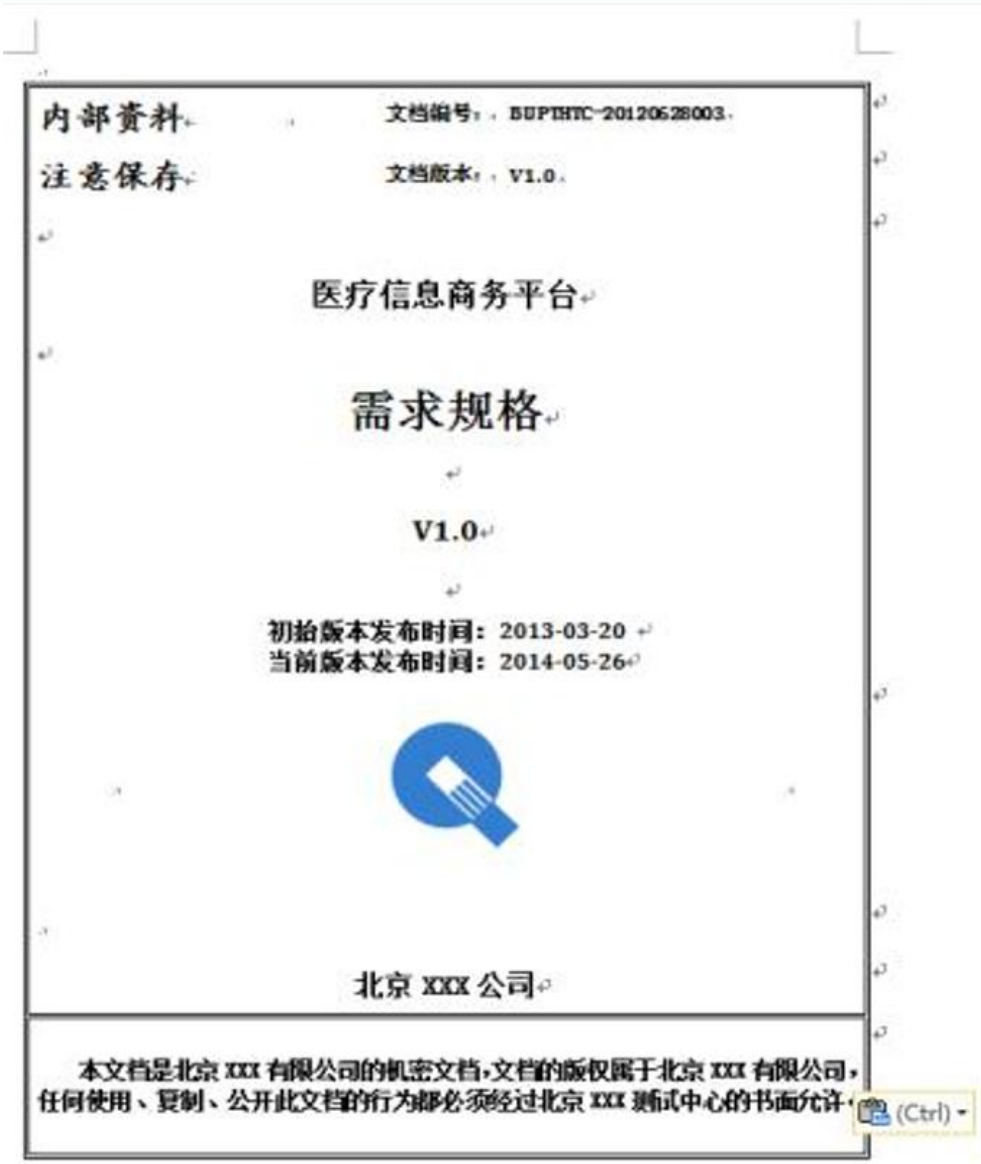
四 AI驱动的需求建模

五 案例分析

MED需求管理案例

- 需求确认：软件需求规格
- 需求变更：变更控制流程

软件需求规格



内部资料
注意保存

文档编号：BUPTHTC-20120528003
文档版本：V1.0

医疗信息商务平台

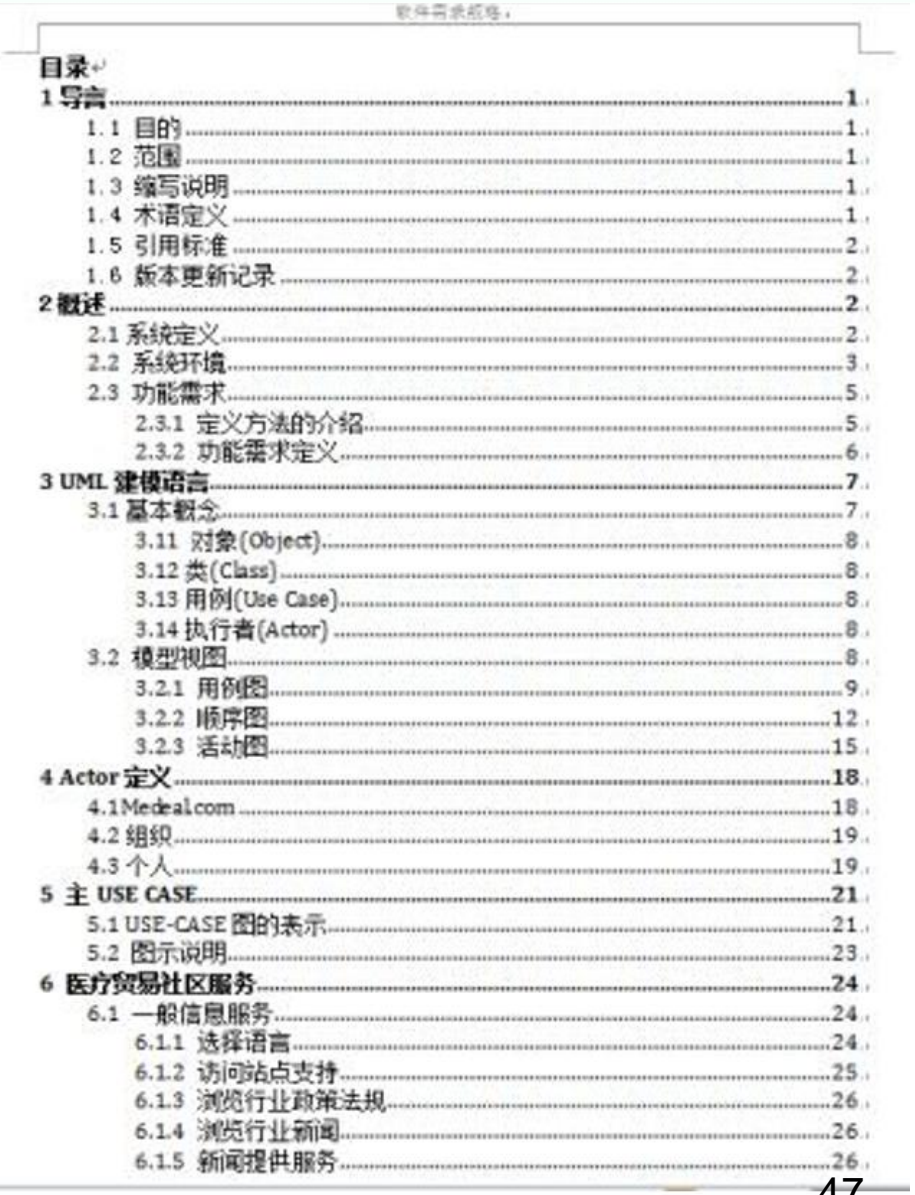
需求规格

V1.0

初始版本发布时间：2013-03-20
当前版本发布时间：2014-05-26

北京 XXX 公司

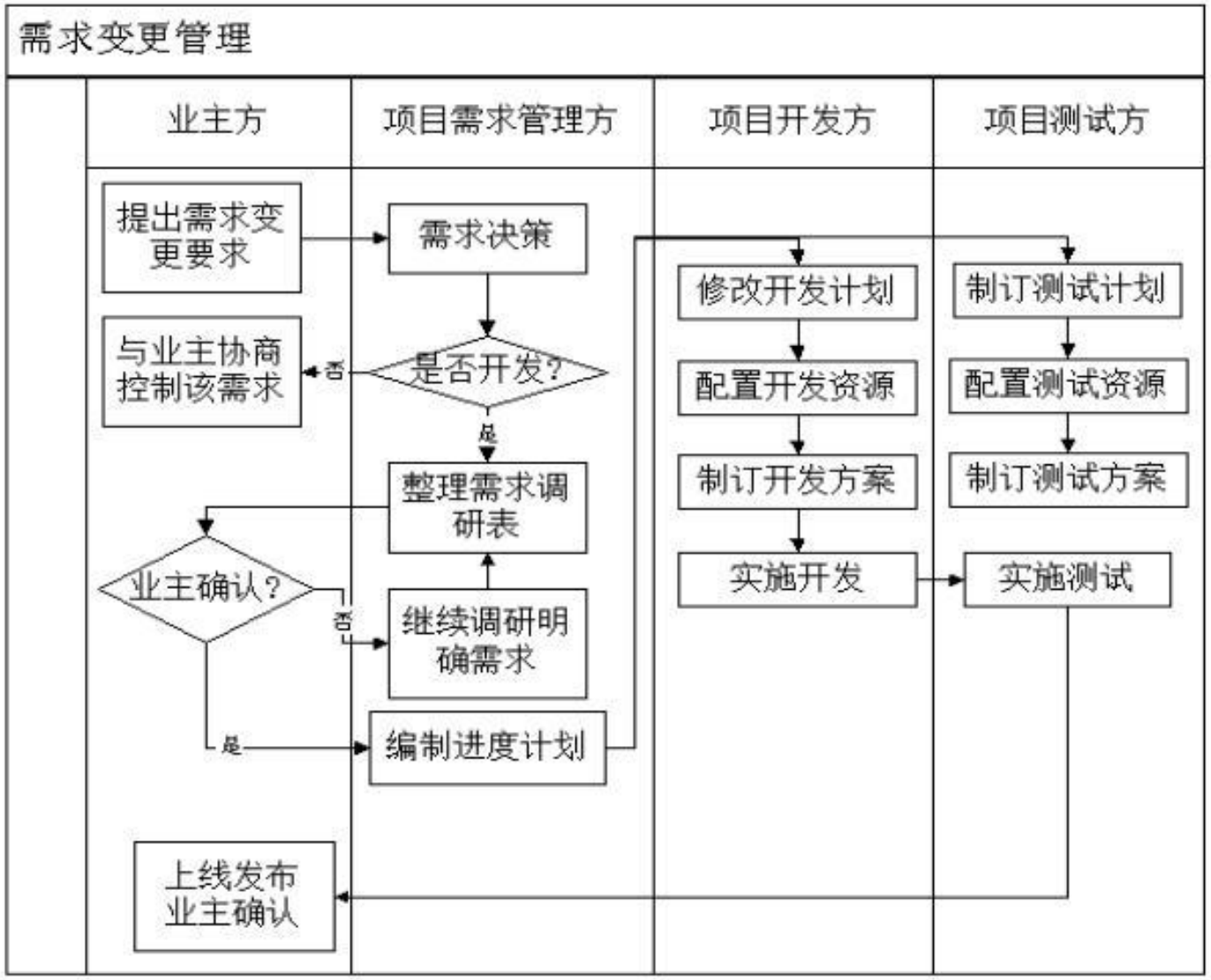
本文档是北京 XXX 有限公司的机密文档，文档的版权属于北京 XXX 有限公司，任何使用、复制、公开此文档的行为都必须经过北京 XXX 测试中心的书面允许。



目录

1 导言 1
1.1 目的 1
1.2 范围 1
1.3 缩写说明 1
1.4 术语定义 1
1.5 引用标准 2
1.6 版本更新记录 2
2 概述 2
2.1 系统定义 2
2.2 系统环境 3
2.3 功能需求 5
2.3.1 定义方法的介绍 5
2.3.2 功能需求定义 6
3 UML 建模语言 7
3.1 基本概念 7
3.11 对象(Object) 8
3.12 类(Class) 8
3.13 用例(Use Case) 8
3.14 执行者(Actor) 8
3.2 模型视图 8
3.2.1 用例图 9
3.2.2 顺序图 12
3.2.3 活动图 15
4 Actor 定义 18
4.1Medeal.com 18
4.2 组织 19
4.3 个人 19
5 主 USE CASE 21
5.1 USE-CASE 图的表示 21
5.2 图示说明 23
6 医疗贸易社区服务 24
6.1 一般信息服务 24
6.1.1 选择语言 24
6.1.2 访问站点支持 25
6.1.3 浏览行业政策法规 26
6.1.4 浏览行业新闻 26
6.1.5 新闻提供服务 26

变更控制流程


需求变更管理
业主方
项目需求管理方
项目开发方
项目测试方
提出需求变更要求
需求决策
修改开发计划
制订测试计划
与业主协商控制该需求
否
是否开发?
配置开发资源
配置测试资源
是
整理需求调研表
制订开发方案
制订测试方案
业主确认?
否
继续调研明确需求
实施开发
实施测试
是
编制进度计划
上线发布业主确认

需求管理过程

- 需求获取
- 需求分析
- 需求编写
- 需求验证
- 需求变更

需求建模的基本方法

- 原型方法
- 基于数据流建模-结构化分析法
- 基于UML建模-面向对象的用例分析法
- 敏捷需求分析

- 传统需求管理

  前提：需求比较稳定

  描述方法：数据流、UML等

  管理：需求变更管理流程

- 敏捷需求管理

  前提：需求不稳定

  描述方法：User Story、BDD等

  管理：优先级迭代式管理